STHYRA CRM · ENGINEERING

Backend Engineering Playbook

A guide for the engineer owning services, data plane, integrations, and infrastructure — from current state to Phase 1 MVP.

For: Backend engineer
Working with: Senior Frontend Engineer

v1.0 · 6-m
Build from

Sthyra CRM — Full-Stack Engineering Playbook

> ***For:*** *A full-stack engineer assigned to take Sthyra CRM from current state to Phase 1 MVP.*

> ***Time to complete:*** *~10 weeks full-time for one engineer, ~4 weeks for a team of 3.*

> ***Read first:*** `README.md` *(this repo),* `STHYRA-SYSTEM-ARCHITECTURE.md` *(AWS plan),* `STHYRA-FEATURES-CATALOG.md` *(every function),* `STHYRA-DEV-FLOWCHARTS.md` *(10 flows).*

This playbook is organized as **9 sequential phases**. Each phase has:

- A clear **definition of done**
- **Concrete tasks** with file paths, code shapes, and test commands
- **Time estimate**
- **Verification checklist** you can run yourself

Use it as a sprint board. Tick tasks as you finish them.

---

§ Phase 0 — Onboarding (Day 1–2)

Definition of done: You can run all tests, boot all services, and read the working code confidently.

› Task 0.1 — Read the master plan and architecture

```
# These are committed alongside the code:
cat README.md
cat STHYRA-PHASE-0-REPORT.md            # 44 pages
cat STHYRA-SYSTEM-ARCHITECTURE.md       # 62 pages
cat STHYRA-FEATURES-CATALOG.md          # 9 pages
cat STHYRA-DEV-FLOWCHARTS.md            # 27 pages

# And the original synthesis:
cat ~/.hermes/plans/2026-08-08_090307-sthyra-crm-visual-intelligence-platform.md
```

Time: 2–3 hours. Don't skip this — every later decision depends on what you learn here.

› Task 0.2 — Set up the dev environment

```
# Prerequisites
node --version          # need 22.22.3+
pnpm --version          # need 11.x
docker --version        # for Postgres
git --version

# Clone + install
cd ~/projects/sthyra-crm
pnpm install

# Build
pnpm build

# Tests
pnpm test
# Expected: 114/114 pass across 8 packages
```

› Task 0.3 — Boot everything locally

```
# Terminal 1: Postgres
docker compose up -d postgres

# Terminal 2: org-service against Postgres
DATABASE_URL=postgres://sthyra-crm:sthyra-crm@localhost:5432/sthyra-crm \
  pnpm --filter=@sthyra-crm/org-service start:pg &

# Terminal 3: project-service
pnpm --filter=@sthyra-crm/project-service start:inmem &

# Terminal 4: user-service
pnpm --filter=@sthyra-crm/user-service start:inmem &

# Terminal 5: membership-service
pnpm --filter=@sthyra-crm/membership-service start:inmem &

# Terminal 6: dashboard
pnpm --filter=@sthyra-crm/dashboard dev

# Smoke test
curl http://localhost:8080/v1/health   # → 200 {"status":"ok"}
curl http://localhost:8082/v1/health
open http://localhost:3000              # dashboard
```

› Task 0.4 — Read the working code in this order

1. `packages/tokens/src/index.ts` — design tokens
2. `packages/observability/src/index.ts` — request-id plugin
3. `packages/auth/src/index.ts` — auth middleware
4. `services/org-service/src/index.ts` — Repository pattern, Region union
5. `services/org-service/src/postgres-repo.ts` — parameterized SQL pattern
6. `services/org-service/src/http.ts` — RFC 7807, Idempotency-Key
7. `services/membership-service/src/http.ts` — `@sthyra-crm/auth` integration
8. `apps/dashboard/src/lib/api.ts` — server-side fetch + request-id

Verification checklist:

- [ ] `pnpm test` shows 114 passing tests
- [ ] `pnpm build` builds all packages
- [ ] All four services boot and respond to `/v1/health`
- [ ] Dashboard renders at localhost:3000

---

§ Phase 1 — Postgres Parity for the Remaining Services (Week 1)

Definition of done: All 4 services have both `InMemoryRepository` *and* `Postgres` `Repository` implementations. Repository is chosen via DI (env var).

› Task 1.1 — User-service Postgres repo

Files to create:

- `services/user-service/src/postgres-repo.ts`
- `services/user-service/src/postgres-repo.test.ts`
- `services/user-service/src/postgres-cli.ts`

Schema:

```
CREATE TABLE IF NOT EXISTS users (
  id TEXT PRIMARY KEY,
  email TEXT NOT NULL,
  display_name TEXT NOT NULL,
  role TEXT NOT NULL,
  org_id TEXT NOT NULL REFERENCES orgs(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (org_id, LOWER(email))
);
CREATE INDEX IF NOT EXISTS users_org_id_idx ON users (org_id);

CREATE TABLE IF NOT EXISTS tokens (
  token_hash TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  org_id TEXT NOT NULL,
  role TEXT NOT NULL,
  issued_at TIMESTAMPTZ NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL
);
CREATE INDEX IF NOT EXISTS tokens_expires_at_idx ON tokens (expires_at);
```

Implementation pattern (copy from `services/org-service/src/postgres-repo.ts`):

1. Define `PgClient` interface (same shape as org-service)
2. Define `PostgresUserRepository implements UserRepository`
3. Use parameterized queries only
4. `UniqueViolationError` on `(org_id, email)` duplicate
5. SHA-256 hashing of tokens at rest

TDD:

1. RED: write `postgres-repo.test.ts` with `FakePgClient` mirroring the org-service pattern
2. GREEN: implement until tests pass
3. REFACTOR: extract shared `PgClient` / `FakePgClient` to a new `packages/pg-test-utils/` if it's getting duplicated

Test command: `pnpm --filter=@sthyra-crm/user-service test` — expect ≥15 passing tests.

› Task 1.2 — Membership-service Postgres repo

Same pattern. Schema:

```
CREATE TABLE IF NOT EXISTS org_memberships (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  org_id TEXT NOT NULL REFERENCES orgs(id),
  role TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, org_id)
);
CREATE TABLE IF NOT EXISTS project_memberships (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL REFERENCES users(id),
  project_id TEXT NOT NULL REFERENCES projects(id),
  role TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (user_id, project_id)
);
```

› Task 1.3 — Wire repository selection by env var

Pattern:

```
class="tok-com">// services/org-service/src/cli.ts
const repo = process.env.DATABASE_URL
  ? new PostgresOrgRepository({ client: pgPool })
  : new InMemoryOrgRepository();
const service = new OrgService(repo);
```

Verification checklist:

- [ ] All 4 services start:pg against Postgres successfully
- [ ] All 4 services start:inmem still work
- [ ] `docker compose down && docker compose up -d postgres` then `pnpm --filter=@sthyra-crm/*-service start:pg` works
- [ ] Integration tests pass with real Postgres in CI

---

§ Phase 2 — Real Authentication (OIDC + SAML) (Week 2)

Definition of done: Users can log in via their employer's IdP (Okta/Entra/Auth0). The `@sthyra-crm/auth` package verifies JWTs locally without an HTTP round-trip.

› Task 2.1 — OIDC discovery + JWKS cache

Files to create:

- `services/user-service/src/oidc.ts` — OIDC discovery, JWKS cache, ID-token verification
- `services/user-service/src/http.ts` — new endpoints:

- `GET /v1/auth/oidc/login?provider={id}` → redirect to provider

- `GET /v1/auth/oidc/callback?provider={id}&code={code}` → exchange, mint Sthyra CRM JWT

- `GET /v1/auth/jwks` → return public JWKS

Pattern:

```
import { Issuer, Client, generators, JWS } from 'openid-client';

export async function discover(issuerUrl: string) {
  return Issuer.discover(issuerUrl);
}

export async function verifyIdToken(
  client: Client, idToken: string, nonce: string,
) {
  return client.validateIdToken(idToken, nonce);
}

class="tok-com">// JWKS cache (jose)
const jwks = createLocalJWKSet(keySet);
const { payload } = await jose.jwtVerify(jwt, jwks, {
  issuer: ISSUER, audience: AUDIENCE,
});
```

› Task 2.2 — Replace `verifyToken` with local JWT verification

File: `packages/auth/src/index.ts`

Replace the HTTP call to user-service with:

```
import { jwtVerify, createLocalJWKSet } from 'jose';
const jwks = createLocalJWKSet(jwksCache);
const { payload } = await jwtVerify(token, jwks, {
  issuer: ISSUER, audience: 'sthyra-crm-api',
});
return {
  userId: payload.sub!,
  orgId: payload['x-sthyra-crm-org'] as string,
  role: payload['x-sthyra-crm-role'] as string,
};
```

Cache JWKS for 10 minutes. Fall back to user-service `/v1/auth/jwks` on miss.

› Task 2.3 — SAML SSO (optional P1)

Files: `services/user-service/src/saml.ts`, admin UI at `apps/dashboard/src/app/orgs/[orgId]/sso/page.tsx`

Accept SAML 2.0 metadata XML upload, provision IdP per tenant, support SP-initiated and IdP-initiated flows.

› Task 2.4 — MFA / step-up auth

For admin actions: TOTP enrollment (`POST /v1/mfa/totp/enroll`) and FIDO2/WebAuthn challenge. On sensitive ops, return `401` with `mfa_required: true`.

Verification checklist:

- [ ] Can log in via OIDC test tenant (e.g., Auth0 dev)
- [ ] Dashboard receives and renders authenticated session
- [ ] `Authorization: Bearer` requests work without round-trip to user-service for verification
- [ ] Token refresh works
- [ ] MFA challenge blocks admin actions without valid TOTP

---

§ Phase 3 — Capture Service (Week 3–4)

Definition of done: A field tester can upload a 360° video from a synthetic source. The pipeline completes (stubbed spatial AI). The dashboard shows the capture status.

› Task 3.1 — Capture-service skeleton

Files to create:

```
services/capture-service/
├── src/
│   ├── index.ts            # CaptureService + Repository
│   ├── postgres-repo.ts    # Postgres implementation
│   ├── http.ts             # Fastify
│   ├── storage.ts          # BlobStorage interface
│   ├── storage/local-fs.ts # dev implementation
│   ├── storage/s3.ts       # prod stub (use AWS SDK)
│   ├── pipeline.ts         # pipeline-run orchestration
│   ├── cli.ts
│   └── *.test.ts
├── package.json
└── tsconfig.json
```

Endpoints:

- `POST /v1/projects/:projectId/captures` — initiate (Bearer JWT, Idempotency-Key, returns upload session)
- `PUT /v1/upload-sessions/:id/chunks/:n` — upload chunk (presigned URL on S3 in prod; local-fs in dev)
- `POST /v1/upload-sessions/:id/complete` — finalize (returns capture ready status)
- `GET /v1/captures/:id` — query status
- `GET /v1/projects/:projectId/captures` — list

Schema:

```
CREATE TABLE IF NOT EXISTS captures (
  id TEXT PRIMARY KEY,
  project_id TEXT NOT NULL REFERENCES projects(id),
  status TEXT NOT NULL,  -- 'uploading' | 'processing' | 'ready' | 'failed'
  client_capture_id TEXT,
  started_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  artifact_urls JSONB,
  UNIQUE (project_id, client_capture_id)
);
CREATE TABLE IF NOT EXISTS upload_sessions (
  id TEXT PRIMARY KEY,
  capture_id TEXT NOT NULL REFERENCES captures(id),
  total_chunks INT NOT NULL,
  received_chunks INT[] DEFAULT '{}',
  sha256 TEXT,
  status TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE TABLE IF NOT EXISTS pipeline_runs (
  id TEXT PRIMARY KEY,
  capture_id TEXT NOT NULL REFERENCES captures(id),
  stage TEXT NOT NULL,
  status TEXT NOT NULL,
  started_at TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  artifacts JSONB
);
```

› Task 3.2 — Pipeline orchestrator (stubbed spatial AI)

File: `services/capture-service/src/pipeline.ts`

Implement the state machine:

```
pending → decoding → sfm → meshing → segmenting → aligning → done
        ↘ failed (any stage error after retries)
```

For Phase 1, **stub** each stage as `setTimeout(100ms); return Promise.resolve({})`. Real spatial AI comes in P1.b.

› Task 3.3 — Mobile-shell upload client (test-only)

File: `apps/mobile-kmm/scripts/upload-test-capture.mjs`

A small Node.js script that:

1. Generates synthetic equirectangular frames (e.g., 30 frames of 1024×512 PNG)
2. Splits into 8 MB chunks
3. POSTs to capture-service
4. PUTs chunks
5. POSTs finalize
6. Polls status

This is your E2E test for the upload pipeline.

Verification checklist:

- [ ] `capture-service` boots in-memory and against Postgres
- [ ] Synthetic capture upload completes
- [ ] Pipeline transitions to `ready` (stubbed)
- [ ] Dashboard shows the capture in the project view
- [ ] Tests: ≥20 for capture-service, all passing

---

§ Phase 7 — AI Copilot (Week 7–9)

Definition of done: A user types "show me open RFIs on Level 3 over $5k" and gets a cited answer with thumbnails.

› Task 7.1 — Copilot service skeleton

```
services/copilot-service/
├── src/
│   ├── index.ts
│   ├── tools/             # tool registry
│   │   ├── query-captures.ts
│   │   ├── spatial-query.ts
│   │   ├── diff-captures.ts
│   │   ├── create-issue.ts
│   │   ├── generate-report.ts
│   │   ├── draft-rfi.ts
│   │   ├── summarize-progress.ts
│   │   └── measure.ts
│   ├── safety.ts          # pre/post-inference pipeline
│   ├── rag.ts             # pgvector hybrid retrieval
│   ├── http.ts            # POST /v1/copilot/query (streaming SSE)
│   ├── cli.ts
│   └── *.test.ts
└── ...
```

› Task 7.2 — Bedrock / LLM integration

Use `@aws-sdk/client-bedrock-runtime` for Claude. Or self-host Llama 3 70B on a GPU node group (Phase 2).

Streaming: use the `InvokeModelWithResponseStream` API. Emit SSE chunks.

› Task 7.3 — RAG with pgvector

```sql
-- Aurora Postgres pgvector
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE embeddings ADD COLUMN embedding vector(1024);

-- Hybrid retrieval: BM25 + dense + reranker
-- Use bge-m3 for embeddings (or OpenCLIP for visual)
```

› Task 7.4 — Mandatory citations + refusal

```
class="tok-com">// Pseudocode
if (!response.citations || response.citations.length === 0) {
  return { refusal: 'I cannot answer without evidence.' };
}
return { delta: response.text, citations: response.citations };
```

› Task 7.5 — Safety pipeline

```
async function safeInfer(req: CopilotRequest): Promise<CopilotResponse> {
  class="tok-com">// Pre-inference
  const redacted = await piiRedactor.redact(req.prompt);
  await promptFirewall.scan(redacted);
  const quarantined = await contextQuarantine.wrap(redacted);

  class="tok-com">// Inference
  const result = await bedrock.invokeModel({ input: quarantined, tools });

  class="tok-com">// Post-inference
  await contentClassifier.scan(result.text);
  const final = await piiInverse.reveal(result.text, requesterTier);
  await c2paStamp.provenance(final, requesterId);

  return final;
}
```

› Task 7.6 — Dashboard Copilot UI

File: `apps/dashboard/src/app/assistant/page.tsx`

Streaming SSE consumer. Renders citations as thumbnails + map pins. Voice mode (Phase 2).

Verification checklist:

- [ ] RFI query returns cited answer within 4 s p95
- [ ] Refusal when no citation possible
- [ ] C2PA provenance stamped on derived images
- [ ] Tenant-pinned: cross-tenant probe returns nothing
- [ ] Red-team suite passes (jailbreak attempts blocked)
- [ ] Hallucination eval passes (faithfulness > 0.9)

---

§ Phase 8 — Integrations (Week 9–10)

Definition of done: A test tenant can connect Procore, ACC, and P6; projects round-trip; RFIs sync bidirectionally.

› Task 8.1 — Integration Hub skeleton

```
services/integration-hub/
├── src/
│   ├── index.ts
│   ├── http.ts
│   ├── sync.ts            # scheduled sync per connection
│   ├── webhook-handler.ts # receives vendor webhooks
│   ├── connectors/
│   │   ├── procore.ts
│   │   ├── acc.ts
│   │   ├── bim360.ts
│   │   ├── p6.ts
│   │   └── ...
│   ├── transform.ts       # vendor → Sthyra CRM type mapping
│   └── *.test.ts
```

› Task 8.2 — OAuth + secret storage

Per-vendor OAuth flow. Store credentials in AWS Secrets Manager. Refresh-token rotation.

› Task 8.3 — Procore first

Most-requested. Implement:

- OAuth connect flow
- Webhook subscription
- Project sync (Procore project → Sthyra CRM project)
- RFI sync (bidirectional)

› Task 8.4 — ACC + P6 next

- ACC: BIM model sync, document sync
- P6: schedule sync, percent-complete sync

› Task 8.5 — Webhook delivery system

File: `services/integration-hub/src/webhook-delivery.ts`

Per dev flowchart §7: HMAC-signed, exponential backoff, DLQ, 3-tier SLA.

Verification checklist:

- [ ] Procore test tenant: project created in Sthyra CRM appears in Procore within 30s
- [ ] ACC test tenant: BIM model uploads sync
- [ ] P6 test tenant: schedule sync with percent-complete updates
- [ ] Webhook delivery with retry + DLQ + audit log

§ Phase 9 — Deploy to AWS (Week 10)

Definition of done: The full stack is live in `us-east-1`, serving real traffic, with monitoring and DR.

› Task 9.1 — Provision AWS infrastructure

Use Terraform (or AWS CDK):

```
# infra/"tok-key">terraform/us-east-1/main.tf
"tok-key">provider "aws" { region = "us-east-1" }

"tok-key">module "vpc" { source = "../modules/vpc" ... }
"tok-key">module "eks" { source = "../modules/eks" ... }
"tok-key">module "rds" { source = "../modules/rds-aurora" ... }
"tok-key">module "s3" { source = "../modules/s3-buckets" ... }
"tok-key">module "elasticache" { source = "../modules/elasticache" ... }
"tok-key">module "opensearch" { source = "../modules/opensearch" ... }
"tok-key">module "secrets" { source = "../modules/secrets-manager" ... }
"tok-key">module "cloudfront" { source = "../modules/cloudfront" ... }
```

› Task 9.2 — Containerize and push to ECR

```
# Per service
docker build -t $ECR/org-service:v0.2.0 services/org-service/
docker push $ECR/org-service:v0.2.0
```

› Task 9.3 — ArgoCD apps

```
# infra/k8s/apps/org-service.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: org-service
  namespace: argocd
spec:
  project: sthyra-crm
  source:
    repoURL: https://github.com/sthyra-crm/org-service-helm
    targetRevision: main
    path: .
  destination:
    server: https://kubernetes.default.svc
    namespace: sthyra-crm
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

› Task 9.4 — Production smoke tests

```
# From a fresh laptop or CI
curl https://app.sthyra-crm.dev/v1/health
# → 200 OK

# Auth flow
curl -X POST https://api.sthyra-crm.dev/v1/auth/oidc/login?provider=okta
# → 302 to Okta

# Issue creation
ISSUE_ID=$(curl -X POST https://api.sthyra-crm.dev/v1/projects/$PROJECT/issues \
  -H "Authorization: Bearer $JWT" \
  -d '{"type":"rfi","body":"test"}' | jq -r .id)
```

› Task 9.5 — Enable monitoring

CloudWatch dashboards, X-Ray traces, PagerDuty routing per master plan §11.

Verification checklist:

- [ ] All services healthy in production
- [ ] SSL Labs scan returns A+
- [ ] SLO dashboards populated
- [ ] DR drill (simulate AZ failure)
- [ ] On-call rotation configured

---

§ Continuous: ship code every day

› Daily workflow

```
# 1. Pull latest
git pull
pnpm install

# 2. Make a branch
git checkout -b feat/copilot-streaming

# 3. Write the failing test FIRST
vim services/copilot-service/src/streaming.test.ts
pnpm --filter=@sthyra-crm/copilot-service test
# Expect: red

# 4. Implement
vim services/copilot-service/src/streaming.ts

# 5. Run tests
pnpm --filter=@sthyra-crm/copilot-service test
# Expect: green

# 6. Typecheck and lint
pnpm typecheck
pnpm lint

# 7. Commit with conventional format
git add -A
git commit -m "feat(copilot): streaming SSE for tool-call responses"

# 8. Push and open PR
git push -u origin feat/copilot-streaming
```

› Per-PR checks

The CI pipeline runs:

1. Lint (ESLint + Prettier)
2. Typecheck (tsc)
3. Unit tests (vitest / node:test)
4. Integration tests (with real Postgres)
5. Build (tsc / next build)
6. SBOM + signed image
7. **Auto-deploy to ephemeral preview env** (4h TTL)

› Per-RC checks

The release-candidate pipeline adds:

- SLO burn rate check
- Lighthouse CI bundle budgets
- Coverage ≥ 80% on changed lines

- Security scan + SBOM diff
- Accessibility audit
- DR drill within 30 days

---

§ Architecture decisions to NOT re-decide

These are pinned in the master plan. Don't second-guess:

Decision	Why
Tenancy data-modeled (every record carries `region`)	Compliance invariant
Repository pattern (InMemory for tests, Postgres for prod)	Standard testing seam
REST + RFC 7807 + Idempotency-Key at the edge	Pinned in §9
Request-ID via `@sthyra-crm/observability`	Every log has request_id
Strict TypeScript (`noUncheckedIndexedAccess`, etc.)	Real bugs caught during dev
Teal+amber palette (not cyan+copper)	Field-safety: amber reserved for warnings
pnpm workspaces (not Yarn/npm)	Already set up; switching costs weeks
monorepo layout: packages/*, services/*, apps/*	Don't add new top-level dirs

§ Risk register (per master plan §15)

Risk	Mitigation
Field-network realism	Device field-test program; Network Link Conditioner in CI
GPU/driver matrix	GPU-fingerprint → known-issue registry
AI hallucination drift	Monthly blind-set refresh; quarterly third-party audit
PII leakage in test data	Nightly PII scan across all test buckets
Multi-region consistency	Monthly failover game day per region
Mobile camera hardware variability	Pin firmware in QA; test staging on N-1 firmware
3DGS generalization on HDR	Auto-fallback to NeRF; per-scene quality gates
Drone regulatory	LAANC at flight-planning; on-staff Part 107 advisor
FedRAMP Moderate timeline (12-18 mo)	Start in P2; StateRAMP as faster interim
Switching cost (OpenSpace imports)	Import tool GA-day-one with documented fidelity

§ Where to get help

- **Stuck on a service pattern?** Look at `services/org-service/` — it's the reference impl
- **Stuck on Postgres SQL?** Mirror `services/org-service/src/postgres-repo.ts`
- **Stuck on tests?** Mirror `services/org-service/src/postgres-repo.test.ts`
- **Stuck on auth?** Look at `packages/auth/src/index.ts` and `services/membership-service/src/http.ts`
- **Stuck on design tokens?** Look at `packages/tokens/src/index.ts`

§ Phase 1 MVP exit criteria (master plan §13)

- [ ] TTF capture < 5 min
- [ ] 5 paid conversions (design partners)
- [ ] NPS > 30
- [ ] 114 tests still green
- [ ] ≥ 80% coverage on changed lines
- [ ] All SLOs green for 7 consecutive days
- [ ] SOC 2 Type II evidence collection started

§ Total time budget

Phase	Scope	Time
0	Onboarding	2 days
1	Postgres parity	1 week
2	Real auth	1 week
3	Capture service	2 weeks
4	360 viewer	1 week
5	Mobile shell	2 weeks
6	BIM viewer	1 week
7	AI Copilot	2 weeks
8	Integrations	1 week
9	Deploy to AWS	1 week

For 3 engineers in parallel: ~4–5 weeks.

End of playbook.